Revisiting Traffic Congestion Control Protocols at the Last Mile

Franziska Friedli

School of Computer and Communication Sciences

Semester Project

June 2025

Responsible
Prof. Katerina Argyraki
EPFL / NAL

Supervisor
Dr. Pavlos Nikolopoulos
EPFL / NAL

Supervisor
Mahdi Hosseini
EPFL / NAL

1 Abstract

Token Bucket Filters (TBFs) are widely used by Internet Service Providers (ISPs) to enforce rate limiting on client connections. This work investigates the interaction between TBF parameters, such as burst size and queue size, and various TCP congestion control algorithms, with a focus on maximizing average throughput. The performance of TCP New Reno and Cubic is tied to their dynamic window behavior, which prevents these congestion control algorithms from achieving an average throughput equal to the TBF policing rate in the presence of small buffers. In contrast, we propose two algorithmic approaches that achieve maximum throughput without requiring large buffers or burst sizes.

Algorithm I estimates the optimal window size as the sum of the Bandwidth-Delay Product (BDP) and the buffer size, minus one Maximum Segment Size (MSS). Algorithm II relies solely on the known policing rate, maintaining a fixed window size of BDP plus one MSS. Notably, Algorithm II sustains a constant sending rate, avoids bufferbloat, and remains robust even under cross-traffic conditions. The findings indicate that, using one of these algorithms, clients can stream at optimal throughput for most of the tested scenarios. For TCP New Reno and TCP Cubic with very small buffers, the algorithms achieve improvements of approximately 33% and 17%, respectively, over the default behavior. Additionally, the proposed algorithms reduce traffic burstiness and provide stable performance across diverse network conditions.

Contents

2 Introduction

Over the past five years, the volume of internet traffic has increased. Video streaming contributes significantly and relies on high-throughput connections. These conditions create a significant challenge for Internet Service Providers (ISPs) to deliver fair and cost-efficient service to all clients [1]. ISPs may therefore decide to throttle or slow down a user's streaming rate, to regulate traffic and clear up network congestion. One common method for enforcing traffic limits is the Token Bucket Filter (TBF), which can be used for both traffic shaping and traffic policing. Traffic shaping prevents packet loss by increasing the queue size at the token bucket, buffering excess traffic. However, this can also increase Round Trip Time (RTT) due to buffer buildup. Traffic policing does not rely on additional buffering. Instead, it enforces limits by immediately dropping packets when the traffic exceeds the allowed rate. The Token Bucket filter transmits packets only when the corresponding amount of tokens are available. It can induce policing by dropping packets when there are not enough tokens available and can enforce shaping by queuing packets that can not be transmitted immediately. To limit a single client's traffic, it can be assumed that the ISP assigns one TBF per client. Although the underlying physical medium may be shared among multiple users, the client is restricted to a specific bandwidth. As long as the client transmits below this bandwidth limit, it creates the abstraction of having a dedicated link, unaffected by other users. When a TCP connection is used between a server and a client, conventional congestion control algorithms such as TCP New Reno and TCP Cubic are not optimized for scenarios where the available bandwidth is exclusively allocated to a single user. These algorithms are designed for shared networks, where traffic from multiple users competes for bandwidth. However, in the presence of a Token Bucket Filter (TBF) that assigns a fixed bandwidth to one client, it should be possible to transmit at a constant rate that fully utilizes the assigned bandwidth. Despite this, traditional TCP congestion control algorithms continue to increase the congestion window beyond the optimal point, eventually causing packet loss. Once loss is detected, they reduce the cwnd abruptly, often more than necessary, leading to under-utilization of the available bandwidth. Since these algorithms rely on detecting packet loss through missing acknowledgments, the feedback loop is delayed. This delay causes packet loss before the window is reduced, negatively affecting performance. The goal of this work is to design a new algorithm that maintains high throughput while avoiding packet loss on a link exclusively used by a single client. The proposed algorithm is designed to transmit at a constant rate that fully utilizes the available bandwidth without triggering loss. To understand the design of this new algorithm, it is essential to analyze the limitations and behavior of existing congestion control mechanisms first.

3 Existing Congestion Control Algorithms

TCP is the transmission control protocol that ensures reliable message delivery. When one packet is lost, it retransmits the lost packet. Once the receiver has received the packet, it sends an acknowledgment. TCP uses a sliding window for flow control which determines the number of packets that the sender can send to the receiver without waiting for recognition packets (ACK packets). This sliding window is continuously adapted depending on the receiver's consumer speed. The receiver advertises a window size that is equal to the available receiver buffer size. If the buffer is full, the receiver advertises a window equal to zero. Congestion Control Algorithms additionally introduce the notion of a congestion window which follows a certain pattern determined by the respective algorithm. To calculate the final sending window, the sender computes the minimum between the congestion window and this advertised window value explained above. Several different congestion control algorithms have been developed over the years for TCP.

3.1 TCP New Reno

TCP New Reno is an extension of TCP Reno, which operates in three main states: slow start, congestion avoidance, and fast recovery. During the slow start phase, the congestion window (cwnd) is increased exponentially. Specifically, the congestion window increases by one Maximum Segment Size (MSS) for each acknowledgment (ACK) received, according to the following formula:

$$\text{cwnd} = \text{cwnd} + \text{MSS}$$

When the congestion window grows beyond the slow start threshold (ssthresh), TCP transitions into the congestion avoidance phase. In this phase, the growth of the congestion window becomes linear rather than exponential. For each ACK received, the congestion window is updated using the following formula:

$$\text{cwnd} = \text{cwnd} + \text{MSS} \times \left(\frac{\text{MSS}}{\text{cwnd}}\right)$$

This ensures that the congestion window increases by approximately one MSS per round-trip time (RTT), leading to a more conservative growth in network utilization. When packet loss is detected through timeout, TCP Reno goes back to

slow start. Finally, when packet loss is detected, through the receipt of three duplicate ACKs, TCP enters the fast recovery phase, allowing it to recover from the loss without reverting to slow start.

TCP New Reno modifies the fast recovery phase of TCP Reno to handle multiple packet losses more efficiently [2]. In standard TCP Reno, the congestion window (cwnd) is reduced by 50% whenever a packet loss is detected through duplicate acknowledgments. If multiple packet losses occur in quick succession, TCP Reno may reduce the congestion window multiple times, once for each detected loss. This can lead to an overly aggressive reduction in the sending rate.

TCP New Reno addresses this limitation by avoiding multiple reductions of the congestion window when several packets are lost during a small enough time window. Instead of reducing the congestion window for each detected loss in this time window, TCP New Reno reduces the congestion window only once. This prevents unnecessary back-off and allows the connection to recover more easily, improving throughput in the presence of multiple packet losses.

After some time, TCP New Reno converges to a steady state where the cwnd oscillates between a maximum value, determined by the network's capacity, and approximately half of that value when a packet loss occurs.

3.2 TCP Cubic

TCP Cubic is a congestion control algorithm designed to improve performance in Long Fat Networks (LFNs) networks with high bandwidth-delay products [3]. Like TCP Reno, it maintains the same three phases: slow start, congestion avoidance, and fast recovery. However, TCP Cubic introduces significant changes in the congestion avoidance phase.

Unlike Reno, which increases the congestion window linearly during congestion avoidance, TCP Cubic uses a cubic function to increase the congestion window:

$$W(t) = W_{\max} + 0.4(t - K)^3$$

With K such that

$$W(0) = 0.7W_{\max}$$

Therefore, the window increases more rapidly when it is far below the previous maximum and more slowly as it approaches that maximum. This allows Cubic to quickly utilize available bandwidth while carefully avoiding congestion near the point where losses previously occurred.

Additionally, TCP Cubic uses a multiplicative decrease factor of 0.7, instead of 0.5 as in TCP Reno. This means that when packet loss is detected, the congestion window is reduced to 70

3.3 TCP BBR

Bottleneck Bandwidth and Round-trip propagation time (BBR) is a congestion control algorithm developed by Google. Unlike traditional loss-based algorithms (such as TCP Reno or Cubic) BBR is delay-based and uses real-time measurements of network conditions such as delivery rate, round-trip time (RTT) and packet loss rate to model the characteristics of the network path and dynamically adjust its sending rate [4]. BBR was designed in response to high-bandwidth networks and the resulting negative impact of bufferbloat. This is a phenomenon where excessive buffering in the network leads to high latency because queuing delay increases. Traditional loss-based congestion control often fills deep buffers before backing off, leading to significant queueing delays. In contrast, BBR attempts to not fill the buffers and tends to be better at avoiding bufferbloat and to operate at the bottleneck bandwidth while maintaining low latency.

BBR offers substantially higher throughput for bottlenecks with shallow buffers or random losses, and substantially lower queueing delays for bottlenecks with deep buffers.

4 Algorithm Design and Description

When the ISP enforces traffic limiting with a TBF, it limits the users sending rate to a specific value with one TBF per client. The underlying physical medium might be shared between clients but each client is assigned a share of the available bandwidth. In this scenario traditional congestion control algorithms can not send at the maximum throughput possible, because of how the congestion window and therefore the sending window evolves. To understand this better, it is useful to observe TCP New Reno's behavior in a simple setting with one client, one fifo-queue in the middle and a sender. In this setup, TCP NewReno exhibits its well-known sawtooth behavior during congestion avoidance. The congestion window increases approximately linearly over time, until a packet loss is detected, at which point it drops to half. On average this yields a throughput of:

$$\text{Throughput} \approx \frac{W_{\max} + \frac{1}{2}W_{\max}}{2 \cdot RTT} = \frac{3}{4} \cdot \frac{W_{\max}}{\text{RTT}}$$

where $W_{\max}$ is the maximum congestion window reached before a loss event, and RTT is the round-trip time. Therefore, it is assumed that TCP Reno achieves only 75% of the theoretically maximum achievable throughput.

The goal is to design an algorithm that can perform optimally in this network setting and achieve the theoretically maximum achievable throughput. The algorithm was designed using the following information: In a non-shared link W_{max} should match the Bandwidth-Delay Product (BDP) plus the size of any buffers present in the network. This is because BDP represents the amount of data required to fully utilize the network link and once the window becomes larger than this value,loss will happen, because the link is overfilled. BDP is computed as follows:

$$\text{BDP} = \text{Bandwidth} \times \text{RTT}$$

This can be explained with a simple analogy. Imagine the network link as a pipe and data packets as water flowing through it. The BDP explains how much water can be in the pipe at once, without overloading it. If the sender transmits less than the BDP, the pipe is not full and the link is underutilized. On the other hand, if more than the BDP plus buffer size is sent, packet loss occurs due to overflow. Because TCP's sending window determines how much unacknowledged data can be on the link, to maximize the throughput, this window should be at least the BDP, otherwise the link is underutilized. As explained before, TCP New Reno does not maintain a window equal to the BDP at all times. Instead, the window oscillates, frequently sending less than the available capacity. Therefore the link is regularly underutilized.

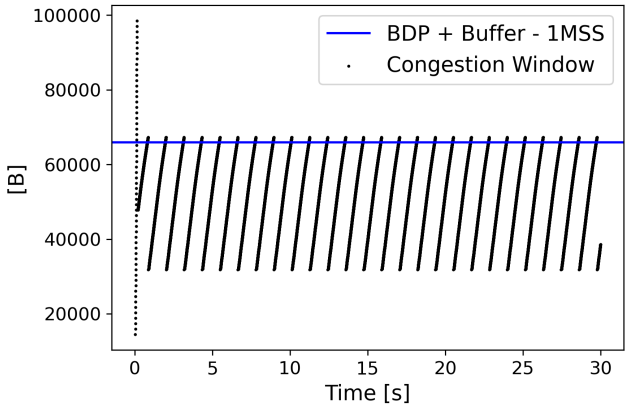


Figure 1. Congestion window evolution for TCP New Reno: The maximum value of the congestion window reaches approximately BDP + buffer size, where BDP is the Bandwidth-Delay Product and the buffer size corresponds to the available queue space at the bottleneck.

The idea behind the Algorithm is to detect and maintain the maximum sending window that does not cause any loss. Instead of increasing the window, beyond the link's capacity and triggering packet loss, the algorithm should stabilize the window just below the loss threshold (blue line in Figure 1), with the idea to ensure continuous high throughput. As can be observed in the figure, for this scenario, this value is indeed the bandwidth delay product plus any buffer present on the network link. Therefore, Algorithm I calculates the estimated maximizing window as:

$$W = \text{Bandwidth} \times \text{RTT} + \text{Buffer Size} - \text{MSS}$$

this accounts for the data that can be carried by the link (BDP), the space in the buffers and subtracts one Maximum Segment Size (MSS) to remain just below the threshold where packet loss would occur.

5 Topology

The network topology is implemented in ns-3 and is rather simple. It consists of a sender node, a receiver node, and a token bucket filter in between them. The token bucket filter is implemented using the ns-3 TbfQueueDisc framework. Tokens are generated at a constant rate and packets are only transmitted when sufficient tokens are available. The number of tokens required to send a packet is equal to the packet size in bytes. Without enough available tokens, the packets have to wait in the queue, or, if there is not enough space left in the queue, packets will be dropped at the end of the queue. All nodes are connected using point-to-point links with a bandwidth of 200 Mbps. To introduce a meaningful bottleneck, the token generation rate is set significantly below this link capacity. The token generation rate is configured to 20Mbps

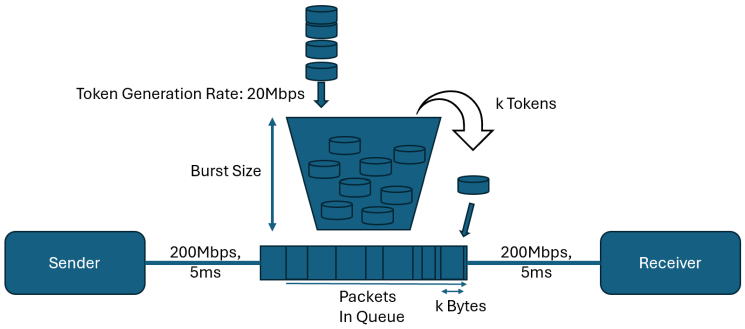


Figure 2. Network Topology

The token bucket filter has additional important parameters that must be defined. One is the burst size, which determines the maximum number of tokens the bucket can hold. This controls how much traffic can be sent in a short burst when the bucket is full or partially full. Another key parameter is the queue size, which specifies how many packets can be held in the queue when there are insufficient tokens to transmit them immediately.

To validate the algorithm in scenarios where the bottleneck is not located at the Token Bucket Filter (TBF), an additional topology was created and simulated. This topology consists of six nodes: two sender nodes, followed by a FIFO queue, a node with a TBF and two receiver nodes. The fourth node has two outgoing network interfaces, one of which connects to a TBF with the same characteristics as used previously. The final two nodes are receivers: one for the primary traffic of interest and the other for cross traffic.

Packets sent by the first sender pass through the FIFO queue, where they are queued together with packets from the second sender, as both streams share the same outgoing link. After the queue, the traffic splits: the first sender's packets pass through the TBF and are delivered to the main receiver, while the second sender's packets (representing cross traffic) are directed to the cross-traffic receiver. Throughput is measured at the receiver that handles the traffic passing through the TBF.

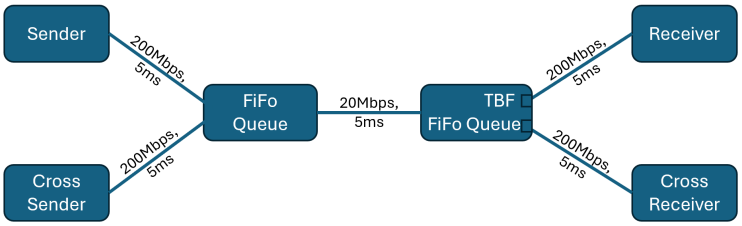


Figure 3. Cross Traffic Topology

During the implementation of the algorithm, several issues were encountered that required modifications to the ns-3 codebase. It was important to understand any internal optimizations or behaviors used by ns-3 that could affect the token bucket filter's functionality and performance.

5.1 NS3 Modifications

The initial approach was to modify the advertised window at the receiver side by using the TCP header's window field to carry the estimated maximizing window, rather than reflecting the actual size of the receiver buffer. However, this method interfered with other implementations and optimizations in ns-3. One of them is the NextSeg() function in the tcp-tx buffer class that works according to [5]. Therefore, the method of advertising the window through the TCP header field for the receiver buffer was soon discarded.

Instead, the value was hardcoded on the sender side as a temporary solution. The sender usually calculates the applied send window as min(cwnd, receiver buffer). This was modified to min(cwnd, receiver buffer, estimated maximizing window by algorithm), to include the estimated maximizing value into the calculation of the minimum. Additionally, since both the sender and the receiver use the same base class for the TCP socket (TcpSocketBase) in ns-3, the sender's

socket was identified in the main function and a Boolean flag was used to activate the new window calculation logic only for the sender. This ensures that the modification does not affect the receiver. However, because the socket is only accessible after the application starts, there is a slight delay of 0.01 s before the new window calculation is applied. To simplify the analysis and understand the algorithm's behavior better, the TCP recovery mechanism was switched from the default in ns-3, that is Proportional Rate Reduction (PRR), to classic recovery.

5.2 Experimental Design

Experiments were conducted using various queue sizes, burst sizes and congestion control algorithms. For TCP Cubic and TCP BBR, the experiments were run under two different link delay conditions: a short delay of 5ms with a simulation time of 30 seconds, and a longer delay of 30ms with a simulation time of 80 seconds. The extended duration in the second case allowed for TCP Cubic to be in the Cubic regime and to observe the evolution of the congestion window as a cubic function. Maximum Segment Size is set equal to 1448 bytes. Two different applications were used during the experiments. The first was the Bulk-Send Application, an existing ns-3 class that continuously attempts to utilize all available bandwidth by sending as much data as possible. The second was a custom Poisson-based application, which exponentially distributes the inter-arrival time of packets while packet sizes are drawn from a cumulative distribution function (CDF). Two different CDFs were used a Google AllRPC CDF and a uniform CDF that selects sizes uniformly at random in the range of [64,1448] bytes 4. A CDF describes the probability that a random variable is less than or equal to a certain value. In the context of packet sizes, the Cumulative Probability at a given packet size tells you the fraction (or probability) of packets that are that size or smaller.

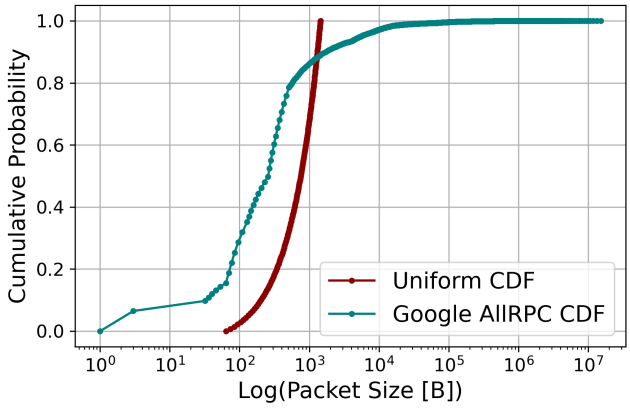


Figure 4. Cumulative Distribution Function (CDF) for Google AllRPC Packet Sizes and Cumulative Distribution Function (CDF) of a Uniform Distribution over Packet Sizes in the Range [64, 1448] Bytes

To establish a baseline, the experiments were always run with the default congestion control behaviour, that does not apply any modification to the TCP window calculation logic. These baseline runs reflect the normal operation of the selected congestion control algorithm. Subsequent experiments introduced the estimated maximizing window which is calculated based on the algorithm described above. The calculation begins by determining the bandwidth-delay product (BDP) and adding the queue size configured for the token bucket filter. From this sum one Maximum Segment Size (MSS) is subtracted. Around this estimated optimal value, a range of window sizes was explored by subtracting and adding up to three MSS. Additionally, a range of the estimated maximizing window around the BDP was tested. From now on, the default case refers to the set of experiments conducted without any window modifications, where the congestion control algorithm operates in its standard. unmodified configuration.

Several different metrics had to be traced in ns-3 during the experiments. The most important values used for analyzing the results include the congestion window, the applied window (i.e., the effective send window after all constraints), and the RX trace at the receiver, which records the accumulated received bytes over time. In addition, number of tokens in the bucket, the number of packets in the queue and the round-trip time (RTT) were monitored to gain a deeper understanding of the system's behavior and for debugging purposes. The primary performance metric used to compare different congestion control algorithms is the average throughput. Throughput is calculated as follows: although the total experiments lasted for typically 30 seconds, the behavior during the initial slow start phase varied between runs due to for example varying burst sizes. As ns-3 fills the token bucket per default in the beginning, when using a large burst

size, initially combinations with large burst sizes will send much more during the slow start phase before loss happens. To ensure fair comparison, throughput was only calculated for the time window after the congestion control algorithm had stabilized. For the shorter experiments (30 seconds), the measurement window started at 5 seconds, for longer experiments (80 seconds), it started at 30 seconds. The throughput was calculated by dividing the accumulated bytes in between those two timestamps by the time duration:

$$\text{Throughput} = \frac{\text{RX}(t_{\text{end}}) - \text{RX}(t_{\text{start}})}{t_{\text{end}} - t_{\text{start}}}$$

where:

$$\begin{aligned} t_{\text{start}} &= 10\,\text{s} \quad \text{(for short experiments)} \\ t_{\text{start}} &= 30\,\text{s} \quad \text{(for long experiments)} \\ t_{\text{end}} &= \text{end of simulation time (e.g., 30s or 80s)} \\ \text{RX}(t) &= \text{accumulated received bytes at time } t \end{aligned}$$

Burst Size [B] (5ms)	Queue Size (5ms)	Burst Size [B] (30ms)	Queue Size (30ms)
1500	1p	1500	1p
3000	2p	3000	2p
4500	3p	6000	4p
6000	4p	10500	7p
7500	5p	13500	9p
15000	6p	25000	16p
22500	7p	43500	33p
25000	8p	50000	39p
30000	10p	73500	69p
37500	11p	103500	99p
45000	12p	133500	129p
50000	16p	163500	150p
55500	20p	193500	170p
70500	24p	223500	190p
85500	28p	253500	210p
100500	32p	283500	230p
115500	33p	313500	
130500	37p	343500	
145500	43p	373500	
160500	49p		
175500	55p		
190500	61p		
205500	67p		
500000			
1000000			

Table 1
Parameter space selection for queue size and burst size for respective link delay

5.3 Expected Improvement

Because it is assumed that the algorithm can keep it's sending window at W_{max} the expected average throughput improvement for the algorithm with respect to default TCP New Reno is :

$$\frac{\frac{W_{max}}{RTT} - \frac{3}{4}\frac{W_{max}}{RTT}}{\frac{3}{4}\frac{W_{max}}{RTT}} = \frac{1}{3} \approx 33\%$$

For Tcp Cubic, the expected improvement will be smaller, as the window is not divided by two everytime a loss occurs but only reduced to $0.7W_{max}$, therefore the improvement is expected to be around 17%.

5.4 Algorithm II

From Figure 7 - 21, which will be discussed further in the results section, a second algorithm was derived. Instead of setting the advertised window to BDP + queue size − 1 MSS, it appears that maximum average throughput may already be achievable by setting the window equal to the bandwidth-delay product (BDP) alone. Therefore experiments were also run for a range around that value. Algorithm II was defined as calculating the estimated maximizing window as BDP + 1MSS.

6 Results

All figures are provided for every combination of congestion control algorithm and application used. Not all figures are shown for all applications in the main body of the document, but can be found in the appendix. The results for the cross traffic topology are presented at the end. The first type of figure consists of several subplots, each corresponding to a fixed queue size. Within each subplot, three curves are shown, representing the evolution across varying burst sizes:

- Black points represent the maximum congestion window observed in the default case (i.e., no modification to the applied window). This value was determined by selecting a time window in which the congestion control algorithm has stabilized and completed at least one full cycle of congestion avoidance. The maximum congestion window within that time frame was then filtered out and displayed.
- Green points show the throughput-maximizing window. However, it only shows the minimum value among all windows that achieved the same maximum throughput for this choice of parameters. For example, a dot at the BDP does not necessarily mean that higher values cannot also yield the same throughput. This happens because the value is picked using the np.argmax function on throughput and then choosing the corresponding window, which picks only the minimum value.
- Blue line represents the algorithm, which estimates the maximizing window at BDP + buffer size - 1MSS
- Purple line represents the BDP.

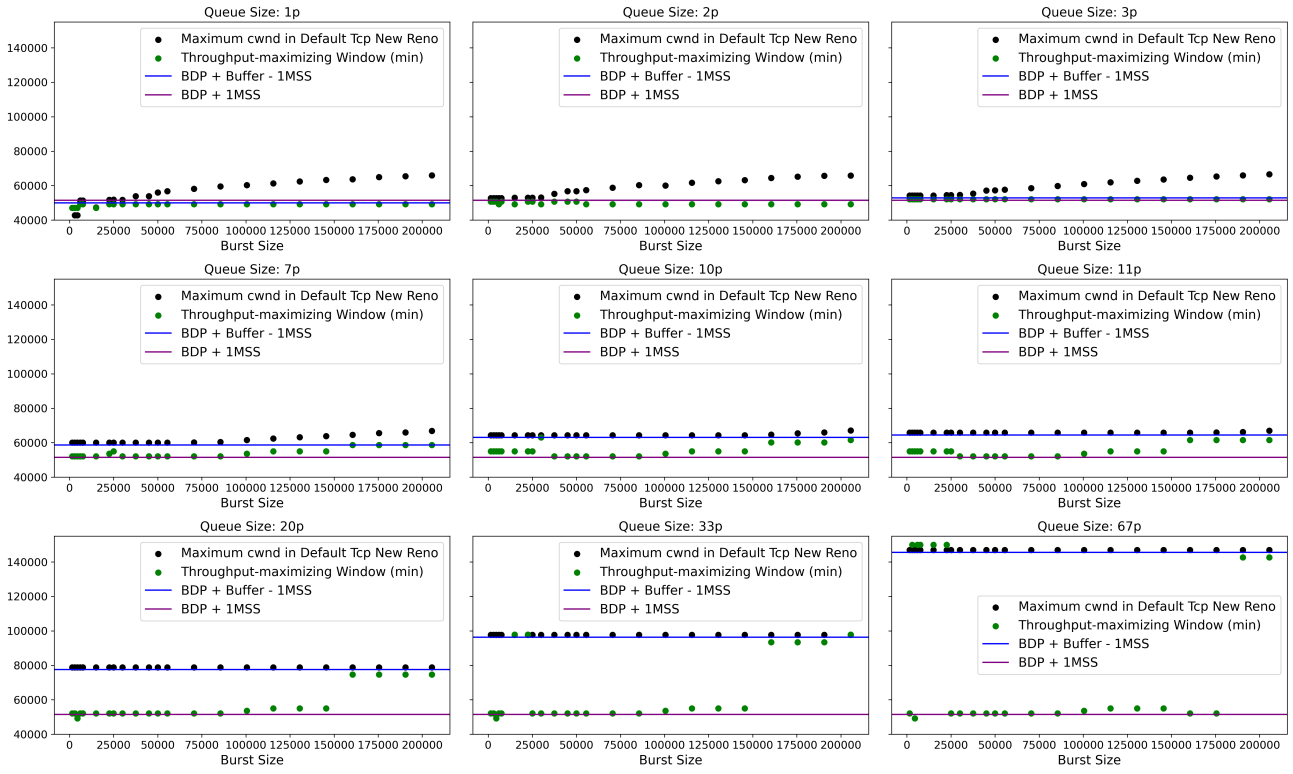


Figure 5. Comparison of Throughput-Maximizing Window, Estimated Maximizing Window, and Maximum Congestion Window in Default TCP New Reno: The blue line corresponds to Algorithm I, and the purple line to Algorithm II. The throughput-maximizing window is observed to lie mostly between these two lines.

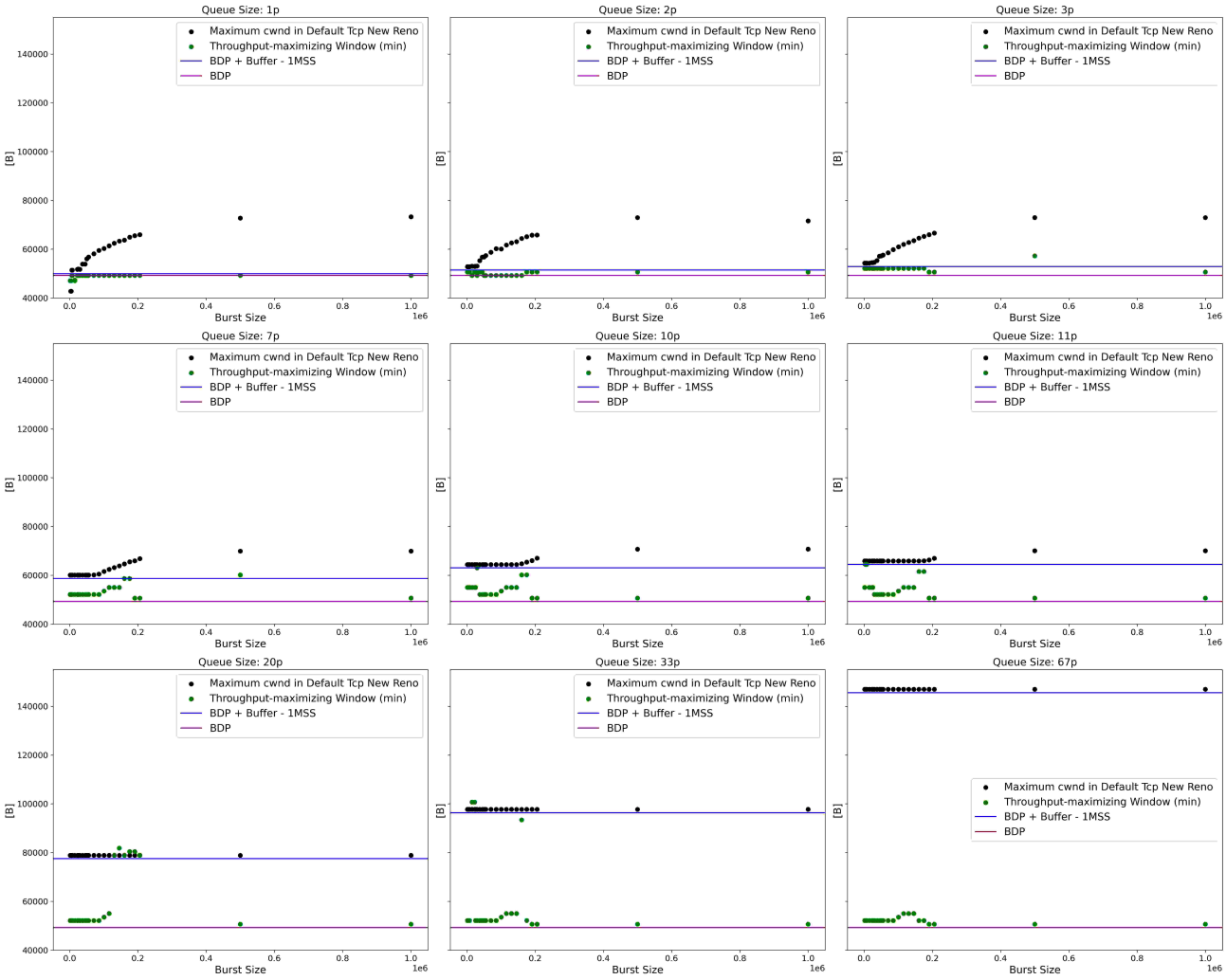


Figure 6. Comparison of Throughput-Maximizing Window, Estimated Maximizing Window, and Maximum Congestion Window in Default TCP New Reno including Burst size 500'000 Bytes and 1MB: The blue line corresponds to Algorithm I, and the purple line to Algorithm II. The throughput-maximizing window is observed to lie mostly between these two lines.

In Figure 5 when the queue size is smaller than or equal to 11 packets, the maximum congestion window increases with increasing burst sizes. However, the throughput-maximizing window does not follow this trend. In most cases, the throughput-maximizing window lies between the purple and blue line, but it does not consistently align with the blue line representing Algorithm I. Another notable observation is that once the queue size becomes sufficiently large, the black points (maximum congestion window values) stabilize and stay at BDP + buffer.

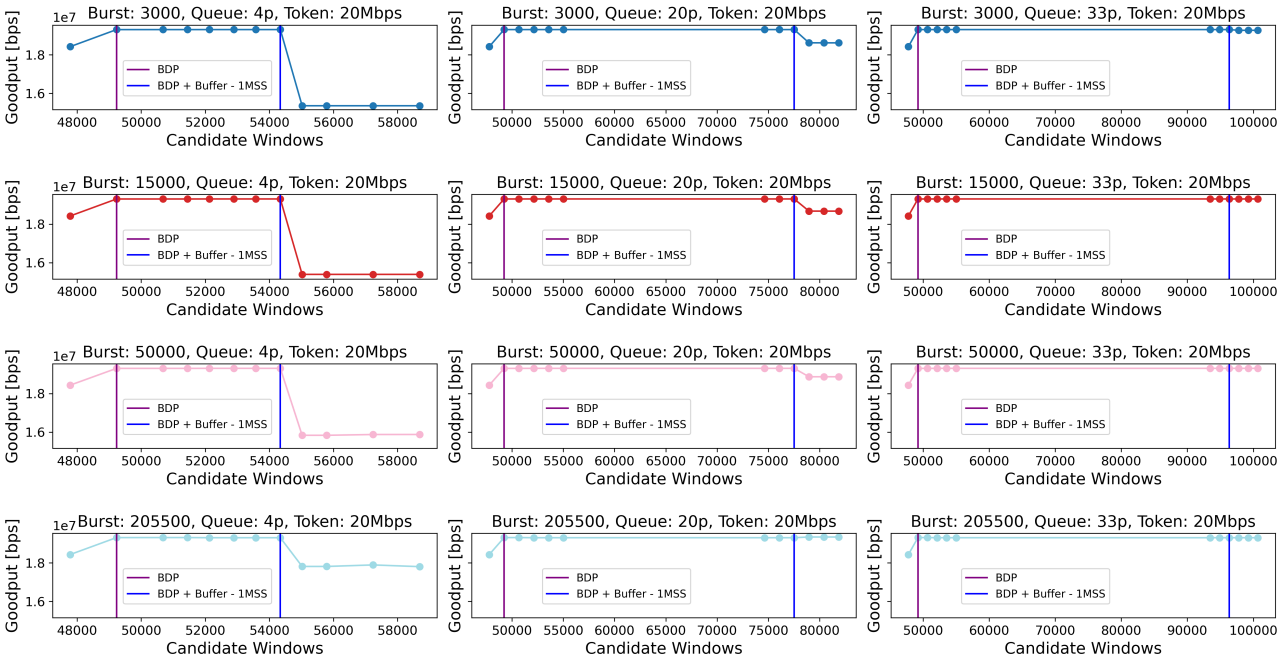


Figure 7. Throughput Evolution over Increasing Estimated Window Values for TCP New Reno with Bulk-Send Application: Each subplot represents a fixed combination of burst size and queue size. Burst size increases from top to bottom, queue size increases from left to right. For each configuration, throughput is plotted against varying estimated maximizing window values. Throughput reaches a maximum over a range of estimated window values.

The second type of figure reveals what was hidden in the first: it may exist multiple throughput-maximizing windows. In this plot, the queue size increases from left to right, while the burst size increases from top to bottom. The y-axis shows the average throughput, and the x-axis represents the estimated window. One can observe that for any given queue size, throughput increases with the window up to a value equal to the BDP. Beyond this point throughput either plateaus or changes only slightly, until the advertised window reaches BDP + buffer - 1MSS. For large queue sizes, throughput becomes almost entirely insensitive to further increases in the advertised window.

The conclusion from those two plots is, that multiple estimated window values can achieve maximum throughput. The initially designed Algorithm is therefore not the only possible choice for an optimal Algorithm. A second algorithm was designed and tested, that estimates the maximizing window at BDP + 1MSS and is called Algorithm II. It is also important to note that the average throughput does not always form a smooth curve for varying estimated windows. This is due to how average throughput is calculated. As I have explained before, the throughput is averaged between two timepoints but those timepoints might be randomly placed in the tcp cycle. As tcp new reno sends data in batches, depending on how batches align with the time window chosen, the throughput might vary minimally and lead to results that are unsignificantly different. As a result, throughput measurements can fluctuate slightly, depending on whether a full batch has just been sent or if the calculation interval cuts through a batch. Lastly, testing with very large burst sizes shows that beyond a certain threshold (e.g. 500'000 bytes and 1MB), the maximum observed throughput saturates and remains constant for all values of estimated windows 6.

The final type of figure is comparing the performance of different algorithms by displaying the average throughput either for a fixed burst size and varying queue size or a fixed queue size and a varying burst size. Average throughput for Algorithm I and Algorithm II is shown in blue and purple, respectively. The green curve represents the maximum throughput achieved among all tested parameters for a given configuration, this indicates the best performance possible

with one of the tested estimated window values. Black triangles indicate the average throughput of the default configuration of the respective congestion control algorithm, while red dots highlight the improvement of Algorithm I over the corresponding default case. In most cases, the average throughput between Algorithm I and Algorithm II is very similar. When they differ, the difference is typically small and may be attributed to throughput calculation noise.

- TCP New Reno: As shown in Figure 8, for TCP New Reno, the expected improvement of approximately 33% is achieved, when the queue size is small. As the queue size increases, this improvement gradually diminishes. Once the queue size reaches the BDP the improvement drops to zero and remains at that level for larger queue sizes.
- TCP Cubic: For TCP Cubic, shown in Figure 9, the behavior is similar to New Reno, except that the improvement starts at around 17% for small queue sizes and decreases as queue size increases. The improvement reaches zero at around half the BDP.
- TCP Cubic, Link Delay = 30ms 11: TCP Cubic for a longer link delay follows the default case behavior for Algorithm I, but does have a similar behavior as for small link delays for small queue Sizes. Looking at the congestion window behavior
- TCP BBR 10: Behaves differently from the loss-based algorithms. With very small queue sizes, BBR's default configuration already performs well and Algorithm I & II show limited improvement. However, as the burst size increases, the improvement increases. In tests with very large burst sizes, the improvement can be substantial.
- TCP BBR, Link Delay = 30ms: In the experiment with larger link delays 12, and therefore a total propagation delay of 120ms, the default configuration already manages to send at a throughput around the TBF policing rate for all the configurations tested.

The results for the other applications were similar and the remaining figures can therefore be found in the appendix.

Last but not least, the cross traffic topology has been tested. The figure shows that for all tested scenarios, the improvement stays at zero.

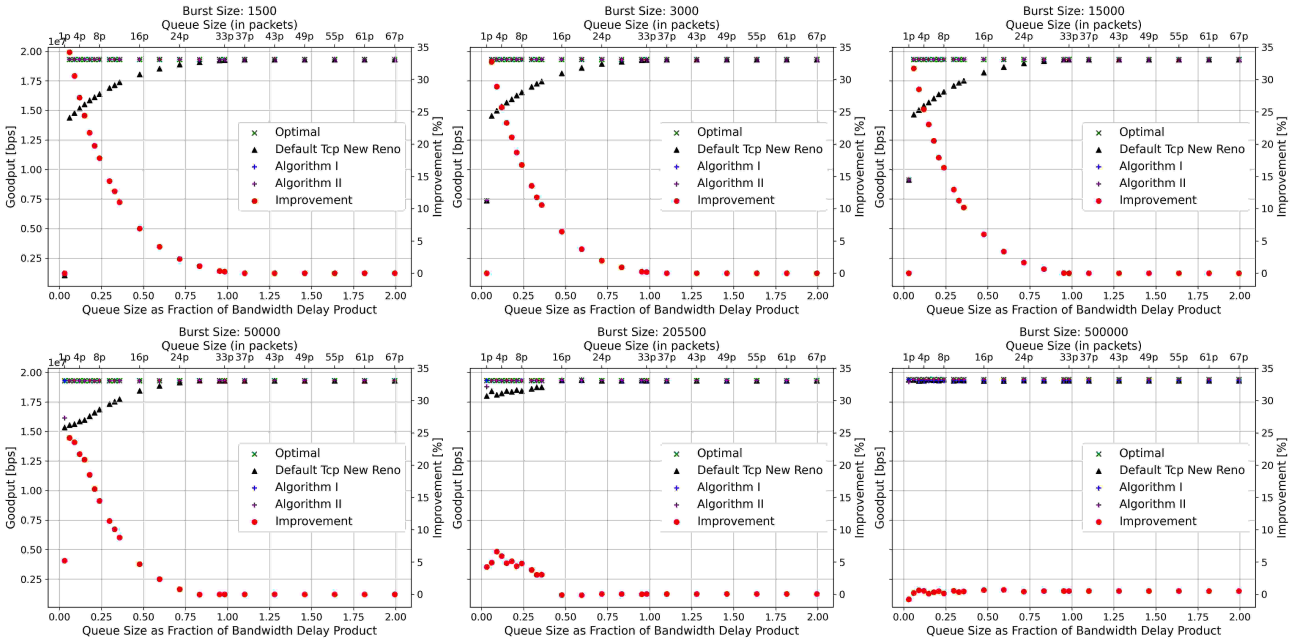


Figure 8. Evaluation of Performance, TCP New Reno with Bulk-Send Application, Link Delay = 5ms: The average throughput is compared across four scenarios: Algorithm I, Algorithm II, the optimal case (using the actual throughput-maximizing window), and the default TCP behavior.

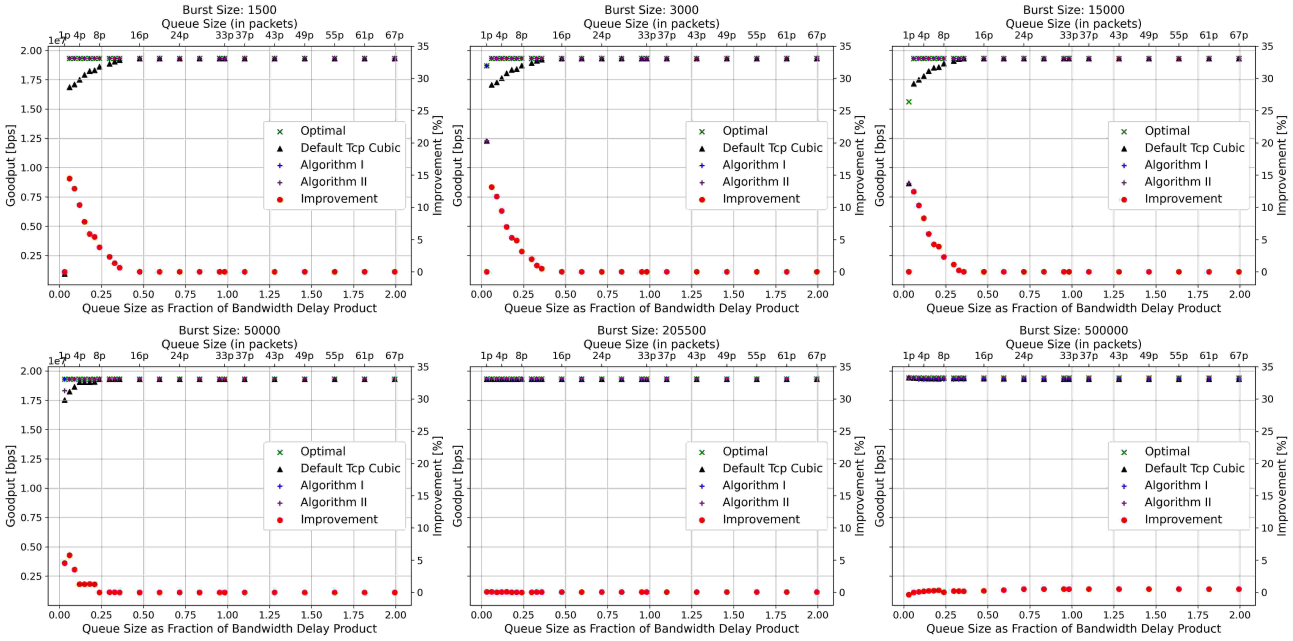


Figure 9. Evaluation of Performance, TCP Cubic with Bulk-Send Application, Link Delay = 5ms: The average throughput is compared across four scenarios: Algorithm I, Algorithm II, the optimal case (using the actual throughput-maximizing window), and the default TCP behavior.

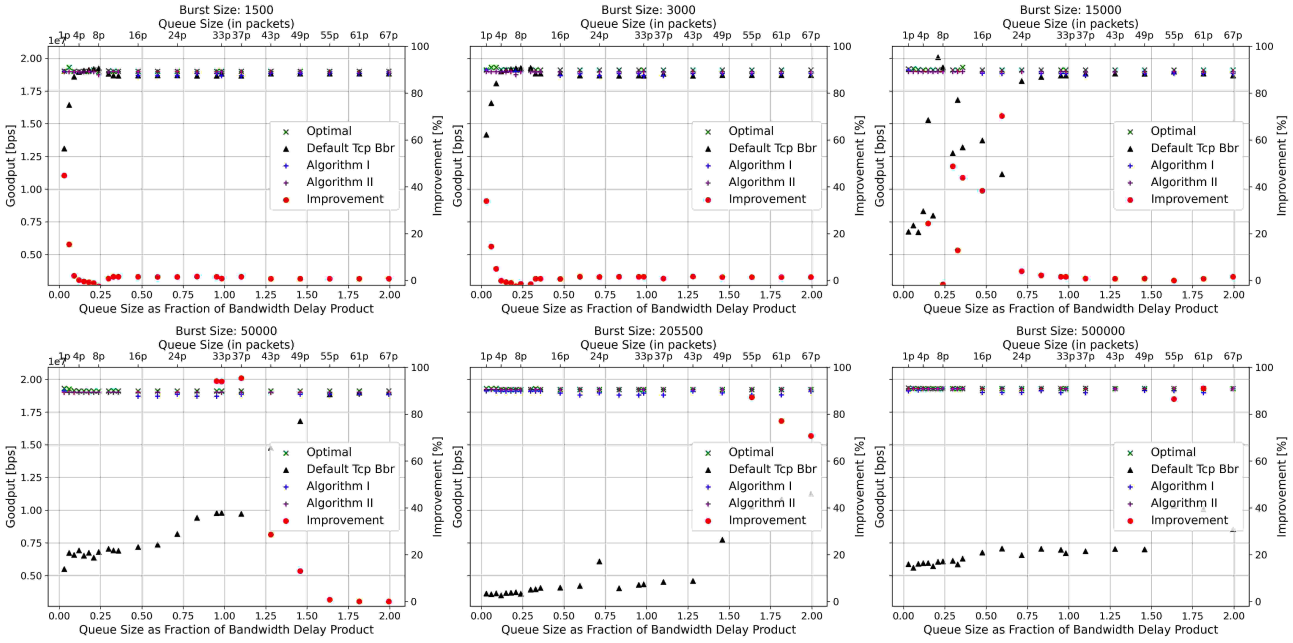


Figure 10. Evaluation of Performance, TCP BBR with Bulk-Send Application, Link Delay = 5ms: The average throughput is compared across four scenarios: Algorithm I, Algorithm II, the optimal case (using the actual throughput-maximizing window), and the default TCP behavior.

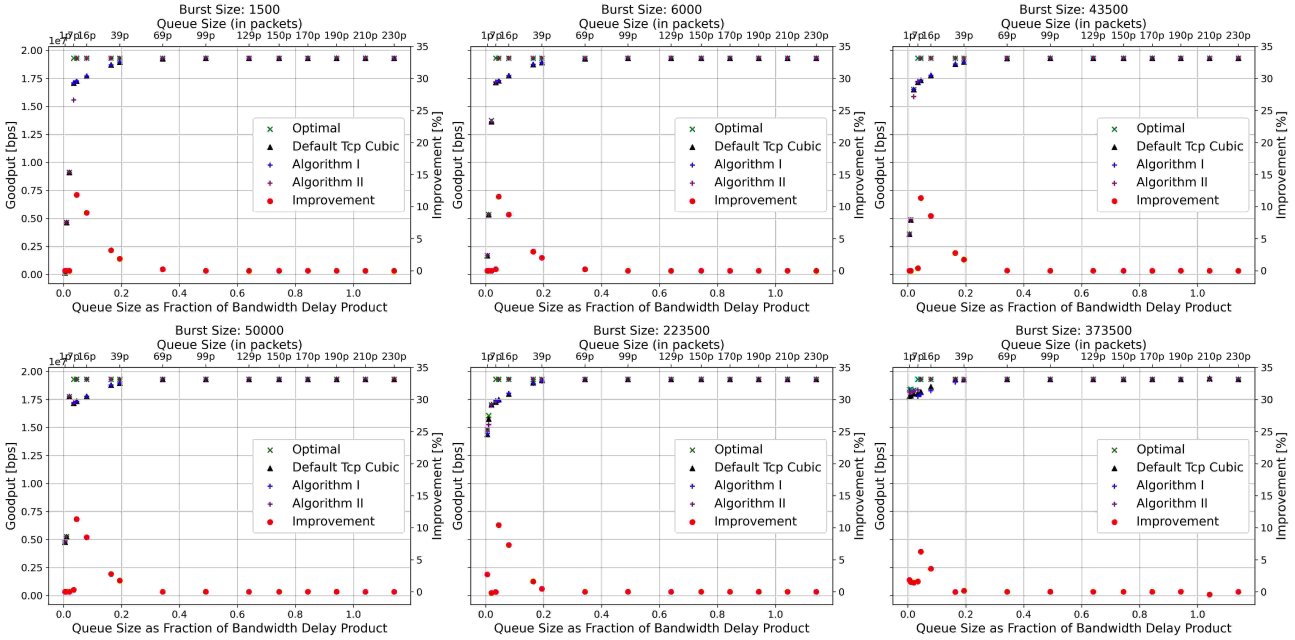


Figure 11. Evaluation of Performance, TCP Cubic, link delay = 30ms, with Bulk-Send Application: The average throughput is compared across four scenarios: Algorithm I, Algorithm II, the optimal case (using the actual throughput-maximizing window), and the default TCP behavior.

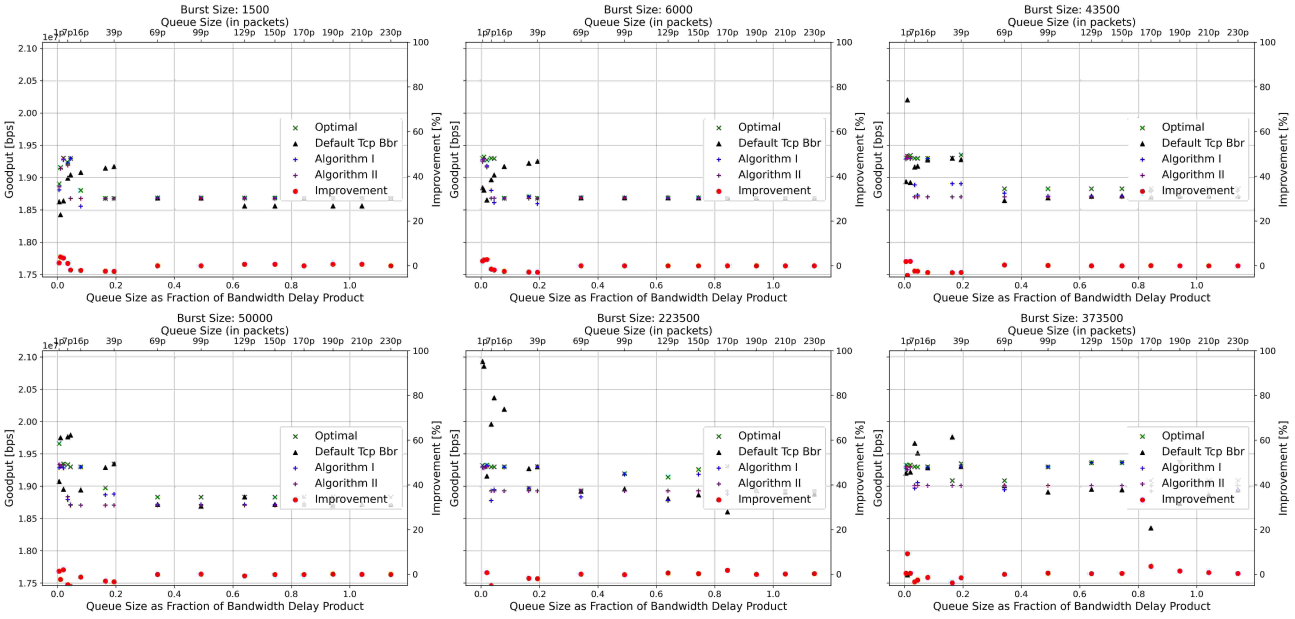
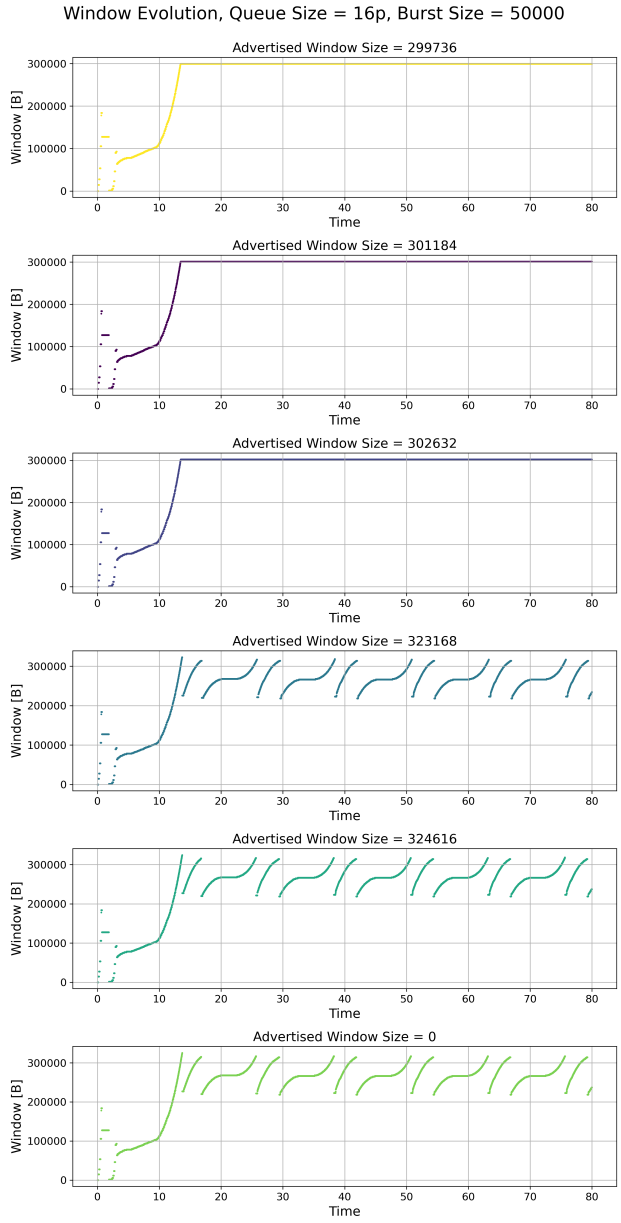
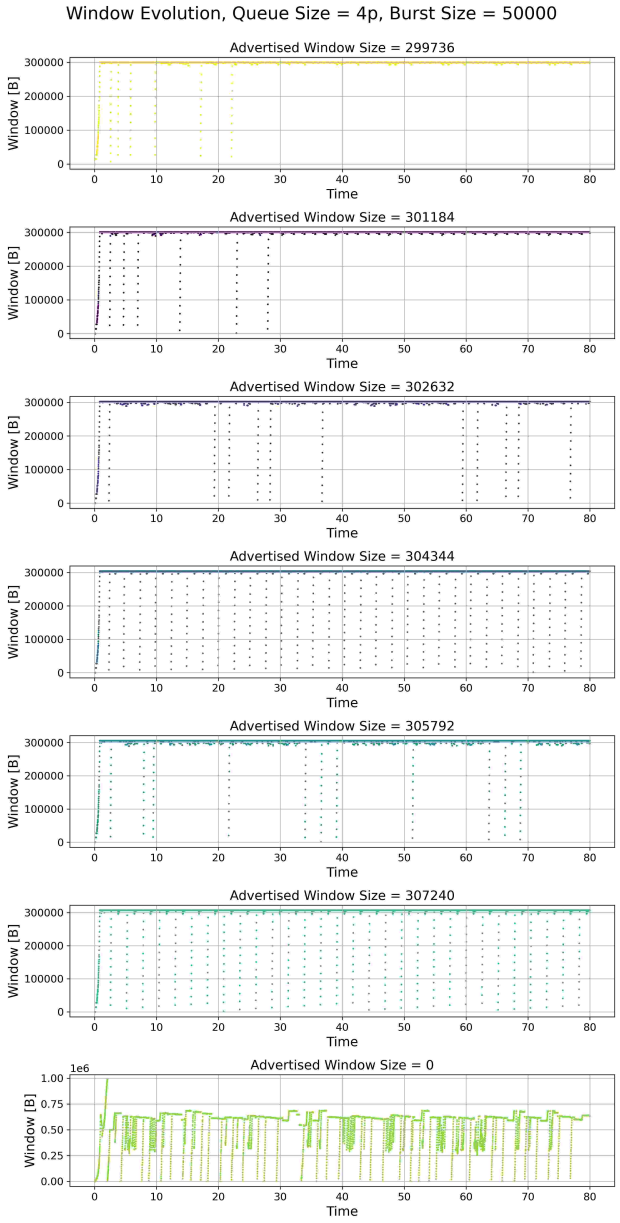


Figure 12. Evaluation of Performance, TCP BBR, link delay = 30ms, with Bulk-Send Application: The average throughput is compared across four scenarios: Algorithm I, Algorithm II, the optimal case (using the actual throughput-maximizing window), and the default TCP behavior.



(a) Applied Window Behavior for Different Estimated Windows in TCP Cubic: Link delay = 30ms, queue size = 16 packets, burst size = 50'000 bytes. The estimated maximizing window computed by Algorithm I is 323'168 bytes, and by Algorithm II is 299'736 bytes. The applied congestion window shows that Algorithm I follows the default TCP Cubic behavior and fails to enforce the calculated window, while Algorithm II successfully applies the estimated maximizing window.



(b) Applied Window Behavior for Different Estimated Windows in TCP BBR: Link delay = 30ms, queue size = 4 packets, burst size = 50'000 bytes.

Figure 13. Applied Window Behavior under TCP Cubic and TCP BBR for Different Estimated Windows.

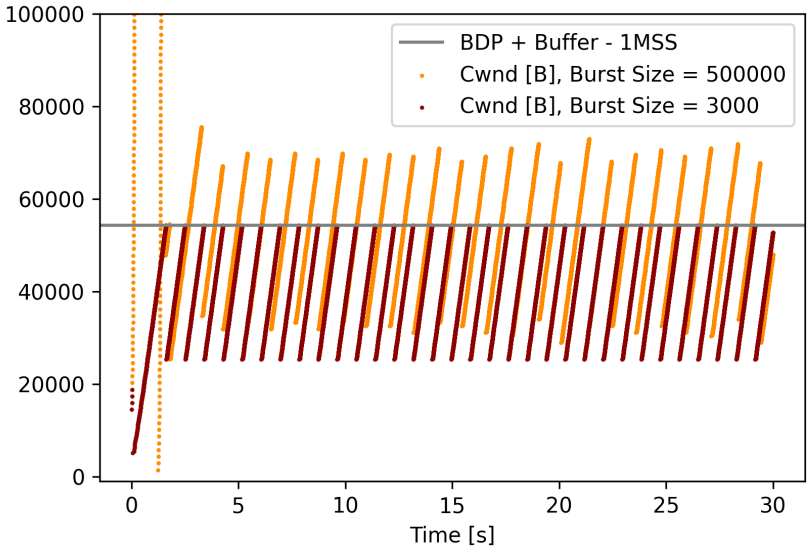


Figure 14. Congestion Window Evolution Comparison for Default TCP New Reno with a Queue Size of 3 Packets and Both Small and Large Burst Sizes: The orange curve, representing the large burst size case, illustrates how the congestion window can exceed BDP + Buffer + 1 MSS, resulting in instantaneous throughput temporarily higher than the TBF policing rate due to token accumulation.

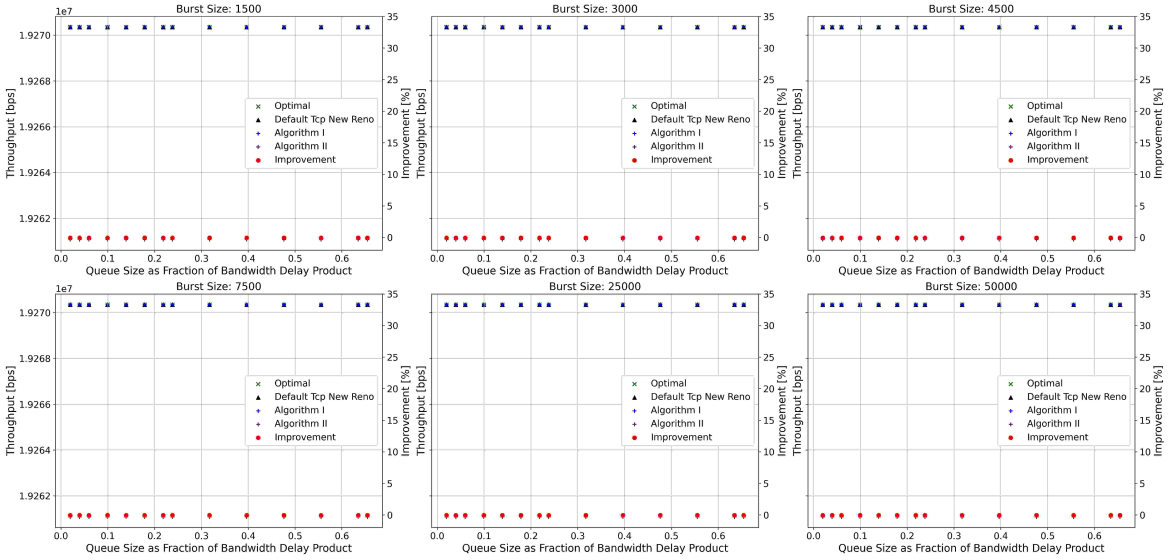


Figure 15. Average Throughput Comparison, Cross Traffic Topology, TCP New Reno: The average throughput is compared across four scenarios: Algorithm I, Algorithm II, the optimal case (using the actual throughput-maximizing window), and the default TCP behavior.

7 Discussion

The experimental results yield several important insights about how parameters such as queue size, burst size, and algorithm design impact congestion control behavior.

7.1 Effect of Queue Size on Congestion Control Algorithm Behavior

Algorithm I and Algorithm II both achieve a throughput equal to the TBF policing rate across nearly all queue size configurations. Remarkably, they maintain this performance even for very small queue sizes, as small as 2 packets for short link delays and 4 packets for long link delays. The only exception is observed with TCP Cubic under large link delays when using Algorithm I: in this case, for small queue sizes, the achieved rate drops to that of the default configuration. As shown in Figure 13a, the congestion window fails to reach the estimated maximizing window and instead evolves similarly to the default case. In contrast, Algorithm II still manages to apply the calculated window correctly. In all other scenarios, both ISPs and clients benefit from the proposed algorithms. Clients can send at maximum throughput even when the ISP would decide to use minimal queue sizes. From the ISP's perspective, the ability to guarantee a sending rate equal to the TBF policing rate, despite small queue sizes, enables resource savings, as queues are implemented in hardware and therefore come with a certain cost.

When queue sizes are large, default TCP New Reno and TCP Cubic can match the algorithms' performance because the buffer can accommodate the size of the data that can fit on the link. The queue never empties, allowing the sender to continuously utilize the outgoing link.

7.2 Effect of Burst Size on Congestion Control Algorithm Behavior

For large burst sizes, default TCP New Reno and TCP Cubic can also reach throughput close to the TBF policing rate. However, their behavior differs from that of the proposed algorithms. In the default case, TCP New Reno exhibits dynamic sending behavior: at times, the link is underutilized, allowing the token bucket to fill up, as tokens are consumed at a smaller rate than the policing rate. As a result, senders can later transmit at instantaneous rates above the policing rate. Larger burst sizes allow more tokens to accumulate, further enabling higher instantaneous throughput. This is illustrated in Figure 14, which compares the influence of small and large burst size configurations on the congestion window. The average throughput can although never exceed the TBF policing rate, even if the burst size becomes very large, because token accumulation stops once the sending rate matches the TBF rate. This imposes an upper limit on the sending window.6.

In contrast, when using Algorithm I or II, the sender transmits at a constant rate equal to the token generation rate. Tokens are consumed immediately and never accumulate. Consequently, increasing the burst size does not influence throughput under these algorithms.

Therefore, although both the algorithms and the default TCP implementations can achieve similar throughput with large burst sizes, they do so for fundamentally different reasons.

As discussed earlier in section 4, TCP BBR behaves differently. For short link delays, BBR does not always reach the TBF policing rate but its performance degrades with larger burst sizes. Because the token bucket is initially full in ns-3, BBR overestimates the sending rate, as it can send more due to the additional tokens in the bucket. It transmits more data than it should, experiences losses, backs off, and repeats this cycle. As the link delay is small the tokens present in the bucket will have a bigger influence on the RTT percentage wise. For larger link delays, BBR estimates the correct sending rate more accurately across all tested burst sizes and is able to reach the policing rate, sometimes even outperforming algorithm I and II. When negative improvement occurs, it although never drops below -5%. This may result from inaccurate throughput measurements or from the behavior of the BBR applied window: Figure 13b shows that, even when Algorithm I or II is applied, BBR's underlying dynamics can cause the actual window to drop below the estimated maximizing window, because the algorithm always selects the minimum of the estimated and calculated congestion window. At the same time, for Algorithm I and II the maximum is still limited by the estimated maximizing window but for default TCP BBR the applied window sometimes goes up to 700'000 which could lead to an overall better average throughput.

Overall, the algorithms function well even with small burst sizes. Since burst size is implemented in software, it likely has negligible implementation cost. However, it is still beneficial to use small burst sizes as this offers reduced interference with other users sharing the same underlying physical medium and therefore increased network stability. While for the default case it is actually beneficial to use a small burst size, when one of the proposed algorithms is applied, it does not really matter, as the sending window will never go above the estimated maximizing window and never allow for instantaneous throughput higher than the TBF policing rate.

It is important to note that the tested network scenario is highly stable, allowing BBR to estimate the sending rate accurately. In more dynamic environment, with for example varying latency, the proposed algorithms may still outperform BBR when BBR fails to estimate RTT accurately.

7.3 Comparison of Algorithm I and Algorithm II

All figure types confirm that Algorithm I consistently achieves throughput equal to the policing rate across all parameter combinations (except for TCP Cubic with a large link delay). However, Algorithm II also achieves this goal while offering key advantages. It prevents buffer build-up, reducing the risk of affecting other flows sharing the same bottleneck, for instance, when a client simultaneously accesses different content platforms. Furthermore, Algorithm II does not require estimation of the queue or burst size; only the policing rate must be known.

7.4 Robustness

Results indicate that as long as the estimated maximizing window falls within the range BDP + 1 MSS to BDP + buffer size − 1 MSS, throughput remains optimal. This tolerance makes the proposed algorithms suitable for real-world networks where the exact BDP or the burst size may be unknown.

7.5 Behavior in Cross Traffic Topology

In the cross-traffic topology, the bottleneck shifts from the Token Bucket Filter (TBF) to a FIFO queue located upstream of the TBF. This is due to the presence of an additional competing sender, which contributes traffic and causes the shared FIFO queue to fill up. Under these conditions, the congestion window never reaches the estimated maximizing window, but when the algorithm is applied, it behaves identically to the default TCP behavior, transmitting at the same rate. This leads to an observed improvement of zero 15, which confirms that the algorithm does not lead to any disadvantage when the bottleneck is not at the TBF.

8 Future Work

In the current implementation, the estimated maximizing window is hard coded at the sender side. However, this is not a scalable or practical approach, as the sender cannot store a unique window value for each receiver. A more realistic solution would involve the receiver communicating the estimated maximizing window at the start of the connection. This could be achieved either by reusing an existing TCP header field, such as the receiver buffer size, or via TCP options. While TCP options may not accommodate the full window value directly, a possible workaround would be to maintain a predefined set of possible window values at the sender side. The receiver could then select an index corresponding to the closest match and communicate that index through a small field in the TCP options.

Additionally, since Algorithm II only requires the policing rate to compute the maximizing window, a key next step would be to explore methods to estimate the policing rate dynamically at the receiver, and integrate that into the window calculation.

Another important analysis could also be to evaluate alternative performance metrics. So far, the primary metric has been total average throughput, but many applications, are more sensitive to latency. This and other metrics should therefore also be considered in future evaluations.

Moreover, a more precise method of calculating average throughput would improve the comparability and accuracy of results across experiments. Finally, it would be interesting to investigate a wider range of link delays to determine the exact threshold at which TCP Cubic diverges from expected behavior under Algorithm I, that is, when the congestion window fails to reach the estimated maximizing window. Identifying this point would help clarify the conditions under which the algorithm is most effective.

9 Conclusion

Two different algorithms have been proposed: Algorithm I estimates the maximizing window as BDP+ Buffer Size - 1MSS, while Algorithm II simplifies the calculation to BDP + 1MSS, relying solely on the policing rate.

Across all tested network parameters, both algorithms almost always achieve a sending rate equal to the TBF policing rate. The exception is TCP Cubic under long link delays, where Algorithm I fails to reach the estimated maximizing window. In contrast, for TCP New Reno and TCP Cubic with short link delays, the algorithms achieve the expected throughput improvements of approximately 33% and 17% respectively for very small buffers. However, as queue and burst sizes increase, the performance of the default algorithms improves, causing the relative improvement of the proposed algorithms to decrease and eventually drop to zero. No negative improvement is observed except in the case of TCP BBR with long link delays, which may result from inaccurate throughput measurements or BBR's underlying dynamic to back off which makes the window drop below the estimated maximizing window.

Overall, the use of either algorithm makes sending behavior more predictable and controlled. From the client's perspective, adopting Algorithm I or II ensures optimal streaming performance for most TBF implementations. From the ISP's perspective, assuming clients use one of the proposed algorithms, the network can guarantee enforcement of the policing rate (and not below), even with the use of minimal hardware resources, potentially reducing implementation cost and improving network stability.

References

[1] Tobias Flach et al. "An Internet-Wide Analysis of Traffic Policing". In: *Proceedings of the 2016 ACM SIGCOMM Conference*. ACM, 2016, pp. 16–29. DOI: 10.1145/2934872.2934873. URL: https://dl.acm.org/doi/abs/10.1145/2934872.2934873.

[2] T. Henderson et al. *The NewReno Modification to TCP's Fast Recovery Algorithm*. Request for Comments 6582. Obsoletes RFC 3782. Category: Standards Track. Apr. 2012. DOI: 10.17487/RFC6582. URL: https://www.rfc-editor.org/info/rfc6582.

[3] Injong Rhee et al. *CUBIC for Fast Long-Distance Networks*. Request for Comments 8312. Category: Informational. Feb. 2018. DOI: 10.17487/RFC8312. URL: https://www.rfc-editor.org/info/rfc8312.

[4] Neal Cardwell et al. *BBR Congestion Control*. Internet-Draft draft-cardwell-iccrg-bbr-congestion-control-02, IETF. Internet-Draft. Work in Progress. Sept. 2022. URL: https://datatracker.ietf.org/doc/html/draft-cardwell-iccrg-bbr-congestion-control-02.

[5] Ethan Blanton et al. *A Conservative Loss Recovery Algorithm Based on Selective Acknowledgment (SACK) for TCP*. Request for Comments 6675. Obsoletes RFC 3517. Standards Track. Internet Engineering Task Force, Aug. 2012. URL: https://www.rfc-editor.org/info/rfc6675.

10 Appendix

Figure 16. Comparison of Throughput-Maximizing Window, Estimated Maximizing Window, and Maximum Congestion Window in Default TCP CUBIC, Bulk-Send Application: The blue line corresponds to Algorithm I, and the purple line to Algorithm II. The throughput-maximizing window is observed to lie mostly between these two lines.

Figure 17. Comparison of Throughput-Maximizing Window, Estimated Maximizing Window, and Maximum Congestion Window in Default TCP BBR, Bulk-Send Application: The blue line corresponds to Algorithm I, and the purple line to Algorithm II.

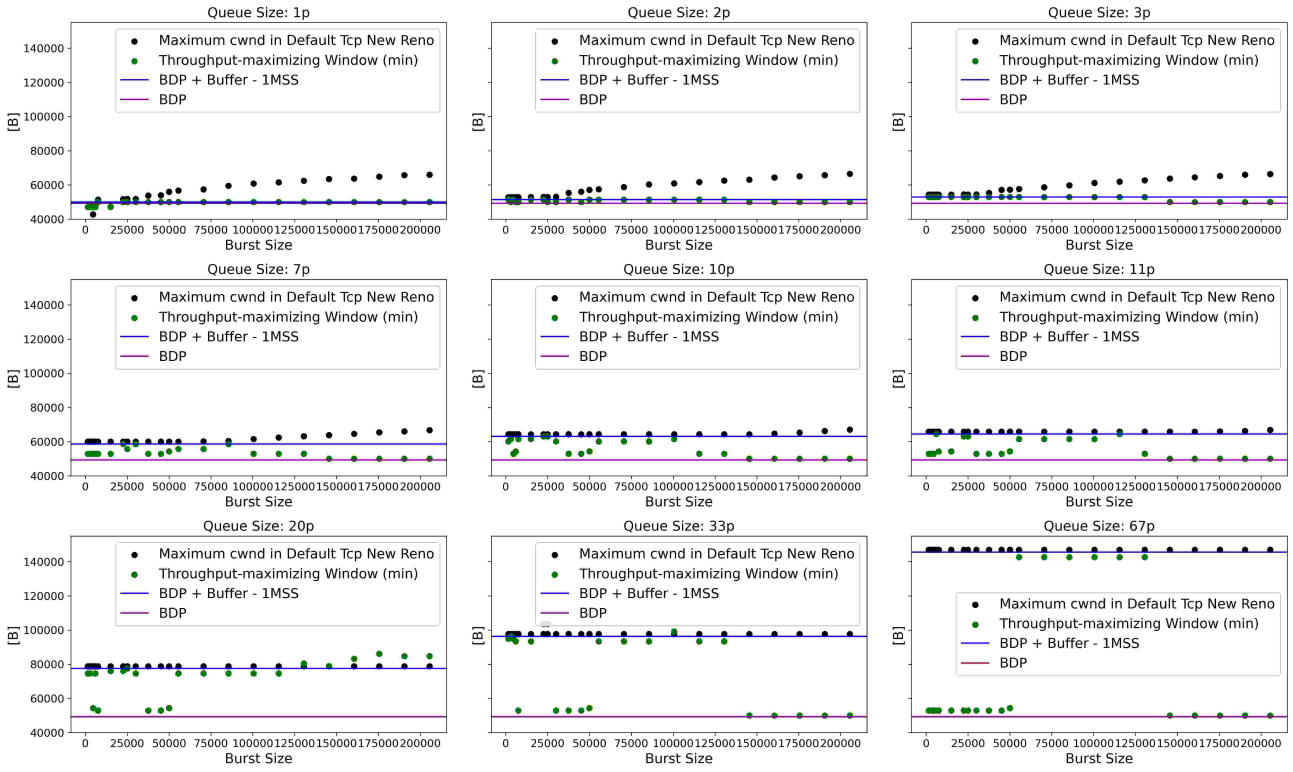


Figure 18. Comparison of Throughput-Maximizing Window, Estimated Maximizing Window, and Maximum Congestion Window in Default TCP New Reno, Poisson Application: The blue line corresponds to Algorithm I, and the purple line to Algorithm II. The throughput-maximizing window is observed to lie mostly between these two lines.

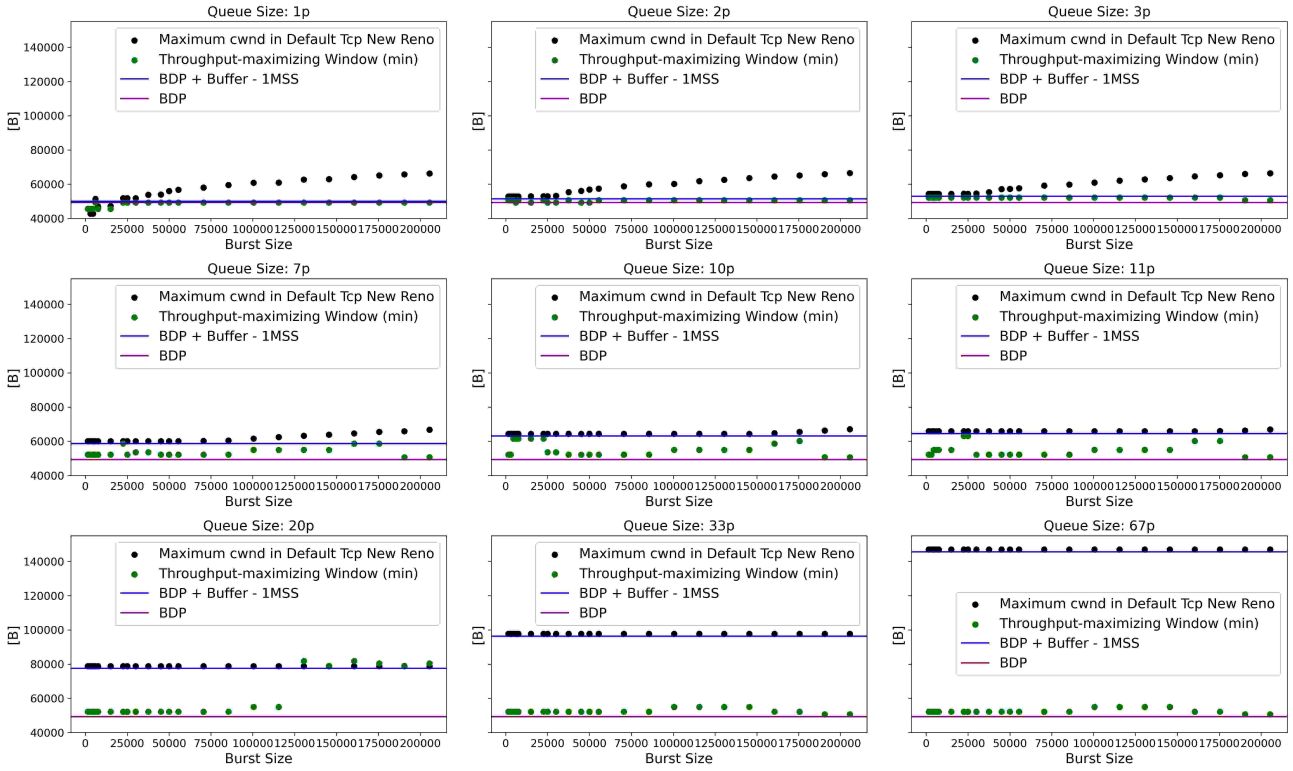


Figure 19. Comparison of Throughput-Maximizing Window, Estimated Maximizing Window, and Maximum Congestion Window in Default TCP New Reno, Uniform Poisson Application: The blue line corresponds to Algorithm I, and the purple line to Algorithm II. The throughput-maximizing window is observed to lie mostly between these two lines.

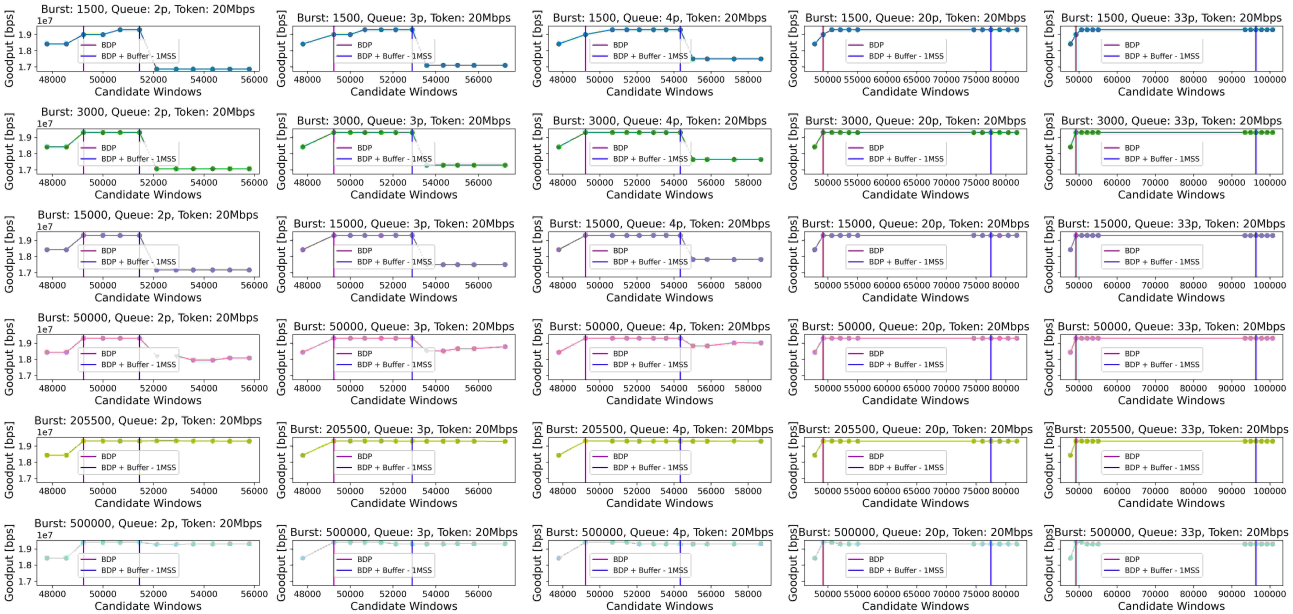


Figure 20. Throughput Evolution over Increasing Estimated Window Values for TCP Cubic with Bulk-Send Application: Each subplot represents a fixed combination of burst size and queue size. Burst size increases from top to bottom, queue size increases from left to right. For each configuration, throughput is plotted against varying estimated maximizing window values. Throughput reaches a maximum over a range of estimated window values.

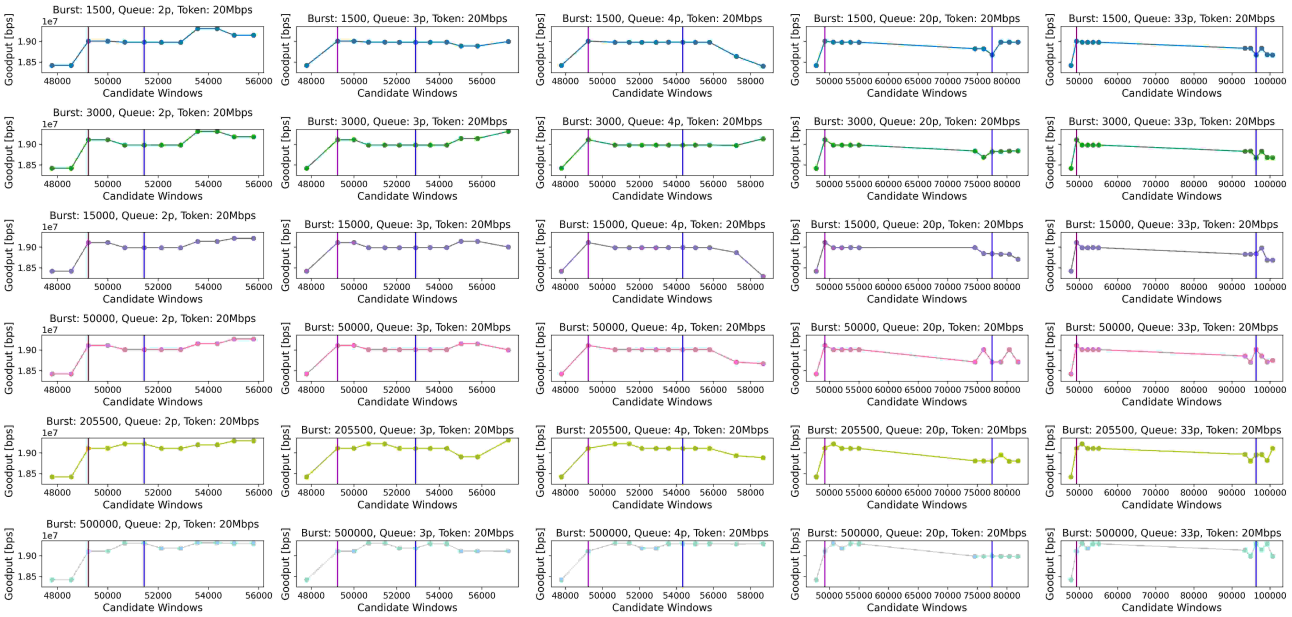


Figure 21. Throughput Evolution over Increasing Estimated Window Values for TCP BBR with Bulk-Send Application: Each subplot represents a fixed combination of burst size and queue size. Burst size increases from top to bottom, queue size increases from left to right. For each configuration, throughput is plotted against varying estimated maximizing window values. Throughput reaches a maximum over a range of estimated window values.

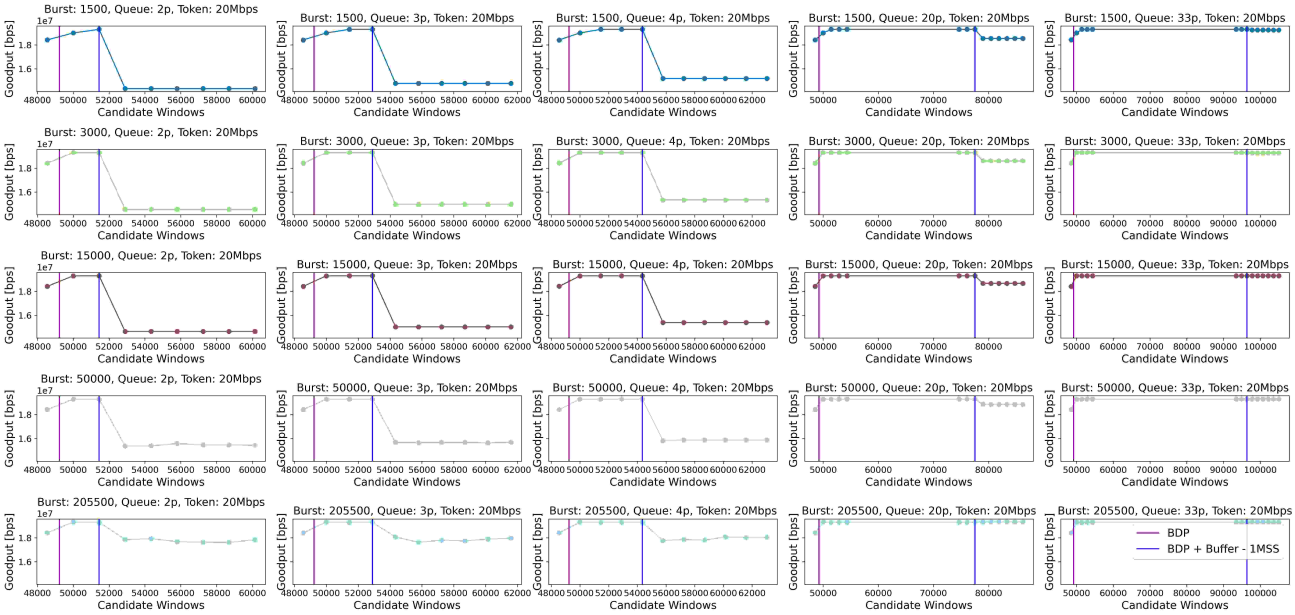


Figure 22. Throughput Evolution over Increasing Estimated Window Values for TCP New Reno with Poisson Application: Each subplot represents a fixed combination of burst size and queue size. Burst size increases from top to bottom, queue size increases from left to right. For each configuration, throughput is plotted against varying estimated maximizing window values. Throughput reaches a maximum over a range of estimated window values.

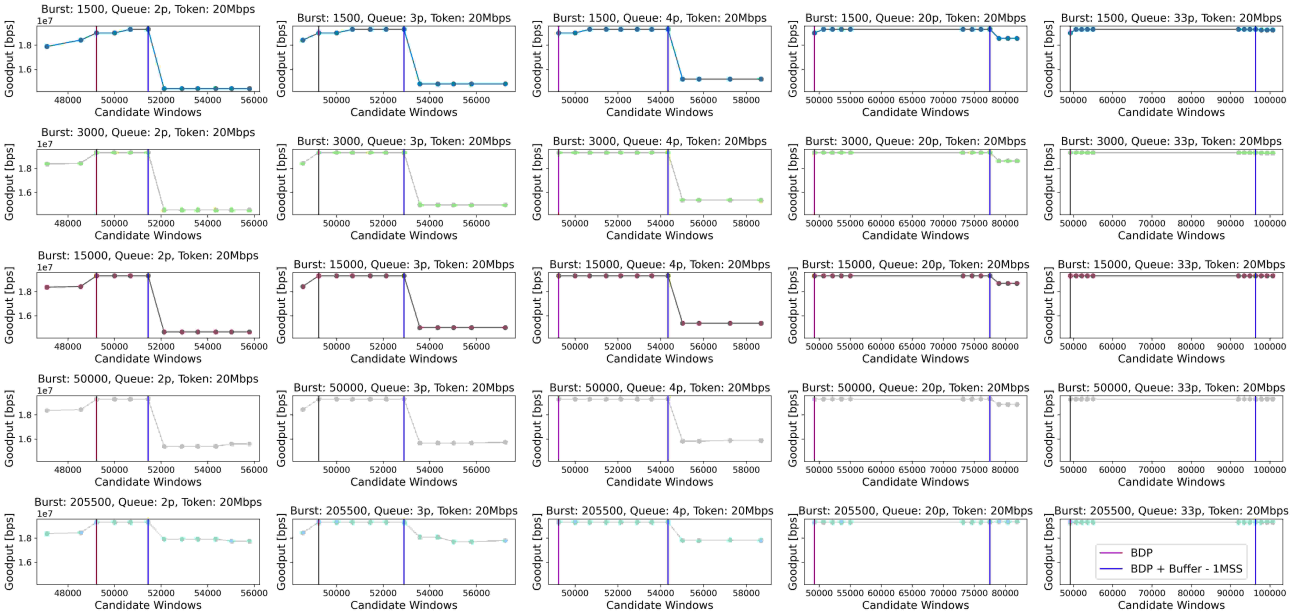


Figure 23. Throughput Evolution over Increasing Estimated Window Values for TCP New Reno with Uniform Poisson Application: Each subplot represents a fixed combination of burst size and queue size. Burst size increases from top to bottom, queue size increases from left to right. For each configuration, throughput is plotted against varying estimated maximizing window values. Throughput reaches a maximum over a range of estimated window values.

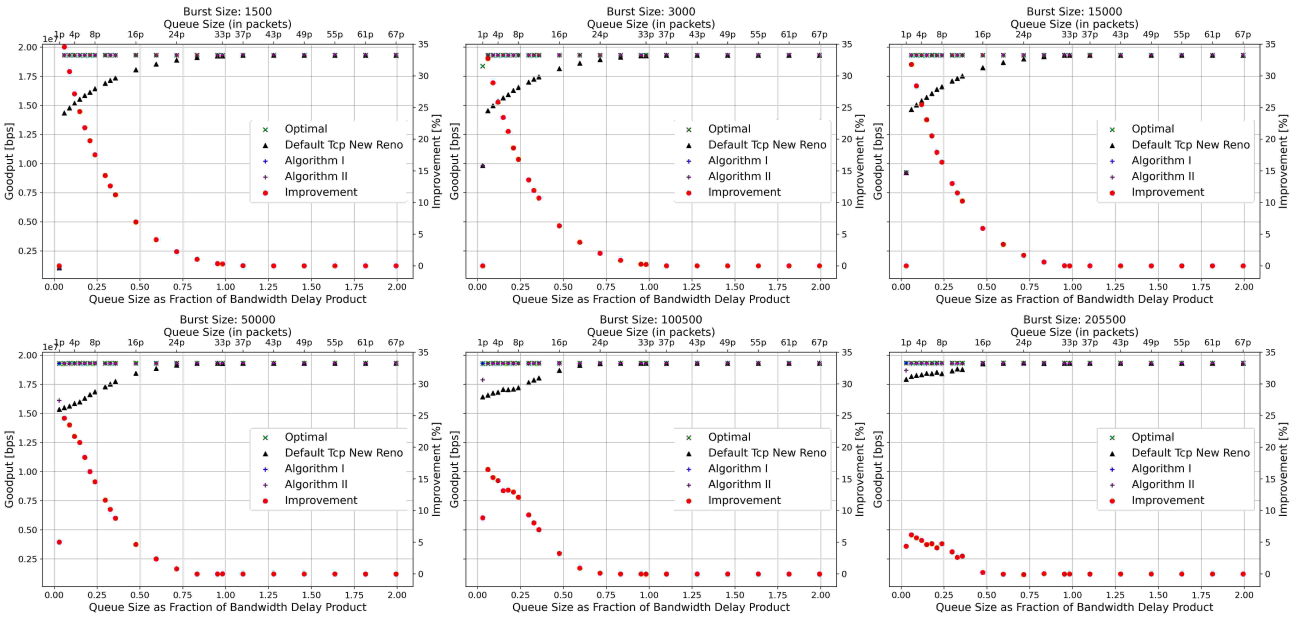


Figure 24. TCP New Reno Poisson Application: Evaluation of Performance. The Average Throughput is shown for Algorithm I, Algorithm II, the optimal case and the Default Case

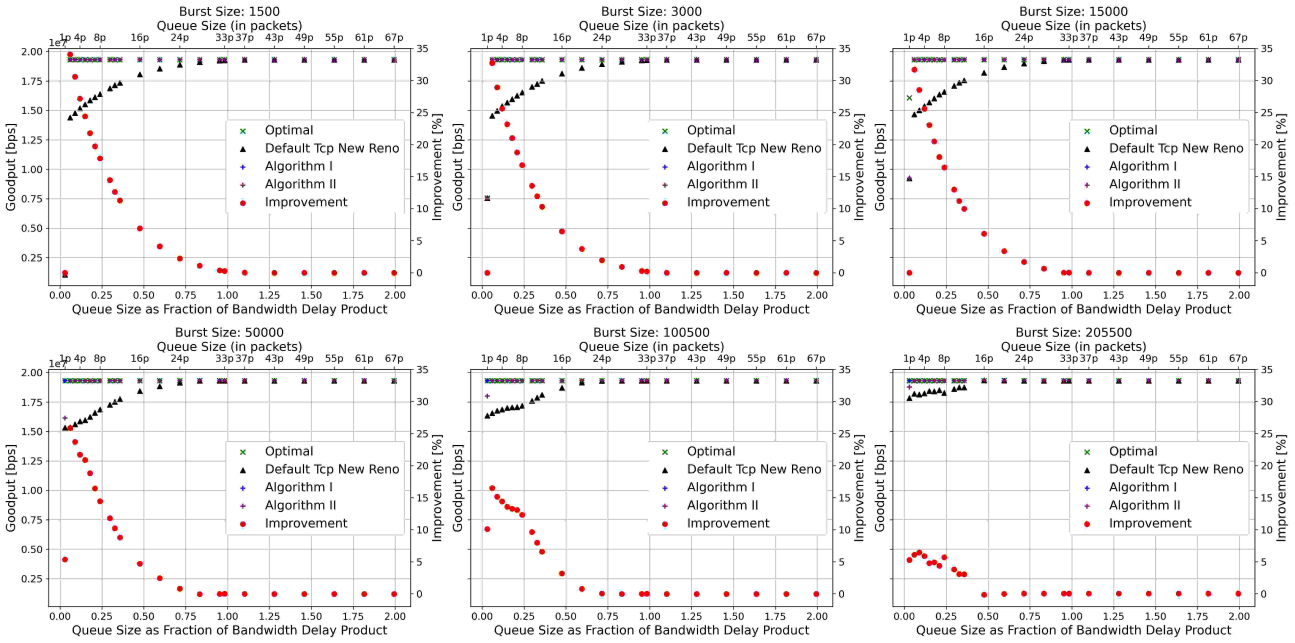


Figure 25. TCP New Reno Uniform Poisson Application: Evaluation of Performance. The Average Throughput is shown for Algorithm I, Algorithm II, the optimal case and the Default Case